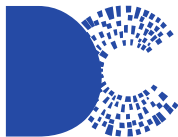




UNIVERSIDADE
FEDERAL DE
SERGIPE



DEPARTAMENTO
DE COMPUTAÇÃO

Organização de memória

Interface Hardware/Software

Bruno Prado

Departamento de Computação / UFS

Introdução

- ▶ Como é organizada a memória?
 - ▶ Os dados são divididos em bytes

ENDEREÇO	BYTE
$0x00..00$	B_1
$0x00..01$	B_2
$0x00..02$	B_3
$\vdots$	$\vdots$
$0xFF..FE$	B_{N-1}
$0xFF..FF$	B_N

Introdução

- ▶ Como é organizada a memória?
 - ▶ Os dados são divididos em bytes

ENDEREÇO	BYTE
$0x00..00$	B_1
$0x00..01$	B_2
$0x00..02$	B_3
$\vdots$	$\vdots$
$0xFF..FE$	B_{N-1}
$0xFF..FF$	B_N

$$4 \text{ GiB} \rightarrow N = 2^{32} = 4.294.967.296 \text{ BYTES}$$

Introdução

- ▶ Tamanho típico dos tipos de dados
 - ▶ Tipos inteiros

Tipo	Bits	Com sinal	Sem sinal
char	8	$-2^7 \leftrightarrow +2^7 - 1$	$0 \leftrightarrow 2^8 - 1$
short	16	$-2^{15} \leftrightarrow +2^{15} - 1$	$0 \leftrightarrow +2^{16} - 1$
int ¹	16 $\leftrightarrow$ 64	$-2^{63} \leftrightarrow +2^{63} - 1$	$0 \leftrightarrow +2^{64} - 1$
long ¹	32 $\leftrightarrow$ 64	$-2^{63} \leftrightarrow +2^{63} - 1$	$0 \leftrightarrow +2^{64} - 1$
long long ¹	64	$-2^{63} \leftrightarrow +2^{63} - 1$	$0 \leftrightarrow +2^{64} - 1$

¹ Valores dependentes da plataforma

Introdução

- ▶ Tamanho típico dos tipos de dados
 - ▶ Tipos inteiros

Tipo	Bits	Com sinal	Sem sinal
char	8	$-2^7 \leftrightarrow +2^7 - 1$	$0 \leftrightarrow 2^8 - 1$
short	16	$-2^{15} \leftrightarrow +2^{15} - 1$	$0 \leftrightarrow +2^{16} - 1$
int ¹	16 $\leftrightarrow$ 64	$-2^{63} \leftrightarrow +2^{63} - 1$	$0 \leftrightarrow +2^{64} - 1$
long ¹	32 $\leftrightarrow$ 64	$-2^{63} \leftrightarrow +2^{63} - 1$	$0 \leftrightarrow +2^{64} - 1$
long long ¹	64	$-2^{63} \leftrightarrow +2^{63} - 1$	$0 \leftrightarrow +2^{64} - 1$

¹ Valores dependentes da plataforma

Padronização por tamanho e sinal em *stdint.h*

Introdução

- ▶ Tamanho típico dos tipos de dados
 - ▶ Tipos reais

Tipo	Bits	Alcance
float	32	$1.2E^{-38} \leftrightarrow 3.4E^{+38}$
double	64	$2.3E^{-308} \leftrightarrow 1.7E^{+308}$
long double ¹	80 $\leftrightarrow$ 128	$3.4E^{-4932} \leftrightarrow 1.1E^{+4932}$

¹ Valores dependentes da plataforma

Introdução

► Big-endian × Little-endian

```
1 uint16_t a = 0xABCD;  
2 float b = 3.14;  
3 int64_t c = 0x12345678ABCD1122;
```

$$a = \{0xAB, 0xCD\}$$

$$b = \{0x40, 0x48, 0xF5, 0xC3\}$$

$$c = \{0x12, 0x34, 0x56, 0x78, 0xAB, 0xCD, 0x11, 0x22\}$$

Introdução

► Big-endian × Little-endian

```
1 uint16_t a = 0xABCD;  
2 float b = 3.14;  
3 int64_t c = 0x12345678ABCD1122;
```

$$a = \{0xAB, 0xCD\}$$

ENDEREÇO	BIG-ENDIAN	LITTLE-ENDIAN
0x00000000	AB	CD
0x00000001	CD	AB

$$b = \{0x40, 0x48, 0xF5, 0xC3\}$$

$$c = \{0x12, 0x34, 0x56, 0x78, 0xAB, 0xCD, 0x11, 0x22\}$$

Introdução

► Big-endian × Little-endian

```
1 uint16_t a = 0xABCD;  
2 float b = 3.14;  
3 int64_t c = 0x12345678ABCD1122;
```

$$a = \{0xAB, 0xCD\}$$

$$b = \{0x40, 0x48, 0xF5, 0xC3\}$$

ENDEREÇO	BIG-ENDIAN	LITTLE-ENDIAN
0x00..00	40 48	C3 F5
0x00..02	F5 C3	48 40

$$c = \{0x12, 0x34, 0x56, 0x78, 0xAB, 0xCD, 0x11, 0x22\}$$

Introdução

► Big-endian × Little-endian

```
1 uint16_t a = 0xABCD;  
2 float b = 3.14;  
3 int64_t c = 0x12345678ABCD1122;
```

$$a = \{0xAB, 0xCD\}$$

$$b = \{0x40, 0x48, 0xF5, 0xC3\}$$

$$c = \{0x12, 0x34, 0x56, 0x78, 0xAB, 0xCD, 0x11, 0x22\}$$

ENDEREÇO	BIG-ENDIAN	LITTLE-ENDIAN
0x00..00	12 34 56 78	22 11 CD AB
0x00..04	AB CD 11 22	78 56 34 12

Introdução

- ▶ Alinhamento dos dados na memória
 - ▶ Dados alinhados

```
1 char a = '@';  
2 uint32_t b = 0xABCD1122;  
3 int16_t c = 0x1234;
```

LITTLE-ENDIAN

0x000000100	40	00	00	00
0x000000104	22	11	CD	AB
0x000000108	34	12	00	00

Introdução

- ▶ Alinhamento dos dados na memória
 - ▶ Dados alinhados

```
1 char a = '@';  
2 uint32_t b = 0xABCD1122;  
3 int16_t c = 0x1234;
```

LITTLE-ENDIAN

0x000000100	40	00	00	00
0x000000104	22	11	CD	AB
0x000000108	34	12	00	00

Preenchimento com zeros (padding)

Introdução

- ▶ Alinhamento dos dados na memória
 - ▶ Dados não alinhados

```
1 char a = '@';  
2 uint32_t b = 0xABCD1122;  
3 int16_t c = 0x1234;
```

LITTLE-ENDIAN

0x000000100	40	22	11	CD
0x000000104	AB	34	12	

Introdução

- ▶ Alinhamento dos dados na memória
 - ▶ Dados não alinhados

```
1 char a = '@';  
2 uint32_t b = 0xABCD1122;  
3 int16_t c = 0x1234;
```

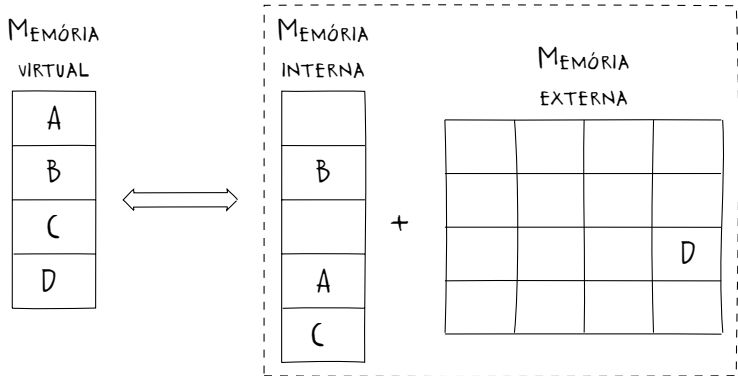
LITTLE-ENDIAN

0x000000100	40	22	11	CD
0x000000104	AB	34	12	

Acesso complexo e lento

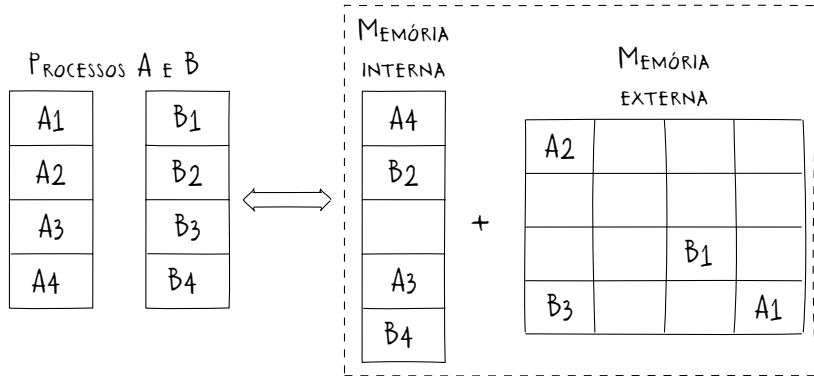
Memória virtual

- ▶ O que é memória virtual?
 - ▶ É uma organização lógica da memória para abstrair os endereços físicos da hierarquia de memória
 - ▶ Cada processo possui seu próprio espaço de endereçamento que é contínuo e dedicado



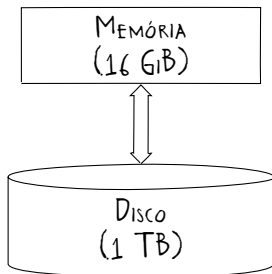
Memória virtual

- ▶ Como a memória virtual é implementada?
 - ▶ A memória física é dividida em blocos chamados de páginas que são alocadas para diferentes processos
 - ▶ Esta organização cria um esquema de proteção dos dados, pois cada processo tem suas próprias páginas



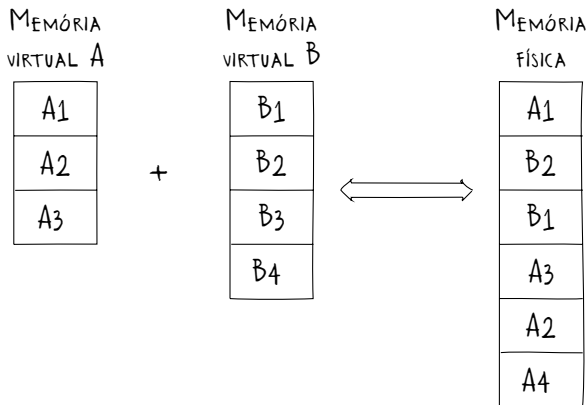
Memória virtual

- ▶ Por que a memória virtual foi desenvolvida?
 - ▶ Para eliminar a responsabilidade do programador de gerenciamento situações onde o software e seus dados não cabem na memória física
 - ▶ Este gerenciamento consiste em dividir o software em partes que são substituídas sob demanda na memória ao longo da execução do software



Memória virtual

- ▶ Por que a memória virtual foi desenvolvida?
 - ▶ Os dados da memória são contíguos e esta organização pode causar fragmentação da memória
 - ▶ Na memória virtual, o mapeamento não contíguo dos endereços evita a fragmentação externa dos dados



Memória virtual

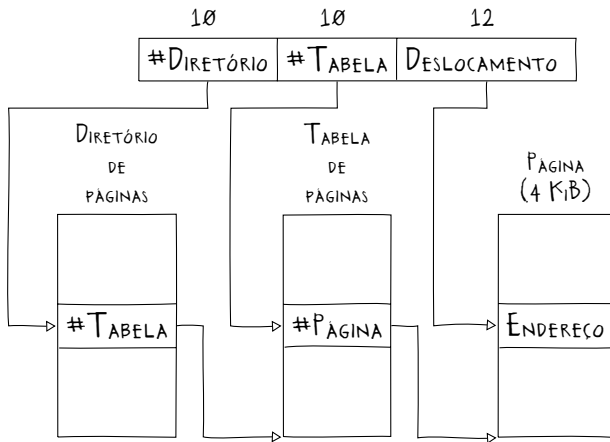
- ▶ Quem faz o gerenciamento da memória virtual?
 - ▶ Hardware
 - ▶ É suportado pela *Memory Management Unit* (MMU), realizando a tradução de endereço, controle de cache e proteção de memória
 - ▶ Software
 - ▶ O sistema operacional mantém na memória uma tabela de páginas para armazenar o endereço físico onde a página está armazenada

Memória virtual

- ▶ Pontos chave do projeto de memória virtual
 - ▶ Utilização de uma técnica associativa de páginas para minimizar a taxa de falta na memória
 - ▶ A melhoria da taxa de faltas pode ser obtida com algoritmos de substituição das páginas
 - ▶ A política de escrita na hierarquia deve ser atrasada (*write back*), uma vez que o acesso dos dispositivos de memória pode ser muito demorado

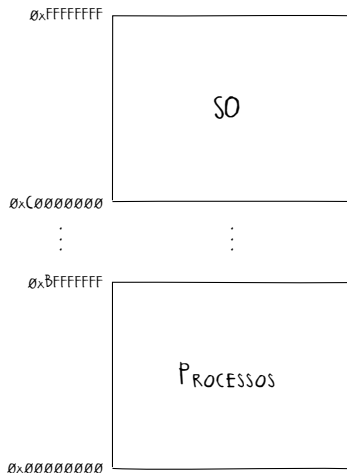
Memória virtual

- ▶ Tradução de endereço virtual
 - ▶ Endereço de 32 bits com página de 4 KiB
 - ▶ Com $2^{32} \div 2^{12} = 2^{20}$ páginas, a tabela de páginas possui até $2^{20} \times 4 = 4 \text{ MiB}$ para cada processo



Memória virtual

► Mapa de memória de 32 bits



Memória virtual

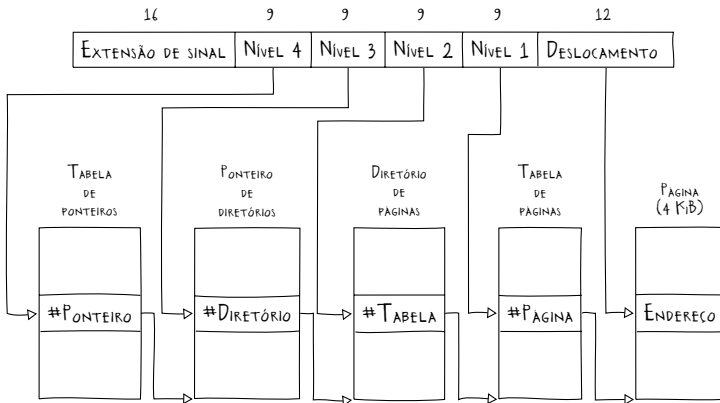
► Mapa de memória de 32 bits



LIMITE DE 4 GiB PARA O TAMANHO DA MEMÓRIA

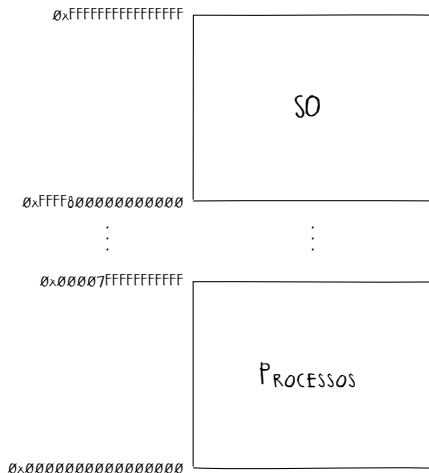
Memória virtual

- ▶ Tradução de endereço virtual
 - ▶ Endereços de 48 bits com página de 4 KiB
 - ▶ Cada nível utiliza uma entrada de 9 bits



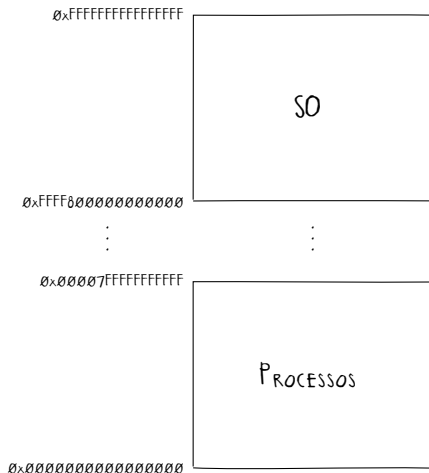
Memória virtual

► Mapa de memória de 64 bits (48 bits)



Memória virtual

► Mapa de memória de 64 bits (48 bits)



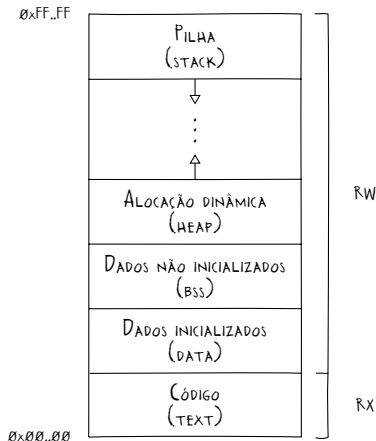
LIMITE DE 256 TiB PARA O TAMANHO DA MEMÓRIA

Exercício

- ▶ Revise os conceitos de hierarquia de memória
 - ▶ Memória cache e *Translation Lookaside Buffer* (TLB)
 - ▶ Políticas de substituição e de proteção
 - ▶ Segmentação paginada
 - ▶ Pesquisa sobre a futura extensão de 57 bits (5 níveis)

Segmentos

- ▶ A memória de cada processo é organizada em segmentos ou seções
 - ▶ Visível ao programador
 - ▶ Proteção de acesso



Segmentos

► Implementação em C

```
1 // Tipos inteiros padronizados
2 #include <stdint.h>
3 // Dado inicializado
4 uint32_t a = 64;
5 // Dado não inicializado
6 uint32_t b;
7 // Função principal
8 int main() {
9     // Deslocamento a direita
10    b = a >> 2;
11    // Alocação na pilha
12    uint16_t c = b - 1;
13    // Retorno de função
14    return c;
15 }
```

Segmentos

► Segmento de código (.text)

```
$ gcc -Wall -g exemplo.c -o exemplo.elf
```

Segmentos

► Segmento de código (.text)

```
$ gcc -Wall -g exemplo.c -o exemplo.elf  
$ objdump -D -z -M intel exemplo.elf
```

Segmentos

► Segmento de código (.text)

```
$ gcc -Wall -g exemplo.c -o exemplo.elf
$ objdump -D -z -M intel exemplo.elf

...
Disassembly of section .text:
...
0000000000000119 <main>:
 1119: 55                push    rbp
 111a: 48 89 e5          mov     rbp, rsp
 111d: 8b 05 05 2f 00 00 mov     eax, DWORD PTR [rip+0x2f05] # 4028 <a>
 1123: c1 e8 02          shr     eax, 0x2
 1126: 89 05 04 2f 00 00 mov     DWORD PTR [rip+0x2f04], eax # 4030 <b>
 112c: 8b 05 fe 2e 00 00 mov     eax, DWORD PTR [rip+0x2efe] # 4030 <b>
 1132: 83 e8 01          sub     eax, 0x1
 1135: 66 89 45 fe       mov     WORD PTR [rbp-0x2], ax
 1139: 0f b7 45 fe       movzx   eax, WORD PTR [rbp-0x2]
 113d: 5d                pop     rbp
 113e: c3                ret
 113f: 90                nop
...
```


Segmentos

- ▶ Segmento de dados
 - ▶ Inicializados (.data)
 - ▶ Não inicializados (.bss)

```
$ gcc -Wall -g exemplo.c -o exemplo.elf  
$ objdump -D -z -M intel exemplo.elf
```

Segmentos

- ▶ Segmento de dados
 - ▶ Inicializados (.data)
 - ▶ Não inicializados (.bss)

```
$ gcc -Wall -g exemplo.c -o exemplo.elf
$ objdump -D -z -M intel exemplo.elf

...
Disassembly of section .data:
...
0000000000004028 <a>:
  4028: 40 00 00          add    BYTE PTR [rax],al
  402b: 00              .byte 0x0

Disassembly of section .bss:
...
0000000000004030 <b>:
  4030: 00 00          add    BYTE PTR [rax],al
  4032: 00 00          add    BYTE PTR [rax],al
  4034: 00 00          add    BYTE PTR [rax],al
  4036: 00 00          add    BYTE PTR [rax],al
...
$
```

Segmentos

- ▶ Segmento de dados
 - ▶ Inicializados (.data)
 - ▶ Não inicializados (.bss)

```
$ gcc -Wall -g exemplo.c -o exemplo.elf
$ objdump -D -z -M intel exemplo.elf

...
Disassembly of section .data:
...
0000000000004028 <a>:
  4028: 40 00 00          add    BYTE PTR [rax],al
  402b: 00              .byte 0x0

Disassembly of section .bss:
...
0000000000004030 <b>:
  4030: 00 00          add    BYTE PTR [rax],al
  4032: 00 00          add    BYTE PTR [rax],al
  4034: 00 00          add    BYTE PTR [rax],al
  4036: 00 00          add    BYTE PTR [rax],al
...
$ exit
```

Segmentos

- Representação dos dados
 - *Executable and Linkable Format (ELF)*

CABEÇALHO
TABELA DE CABEÇALHO DO PROGRAMA (OPCIONAL)
SEÇÃO 1
⋮
SEÇÃO N
⋮
TABELA CABEÇALHO DA SEÇÃO

CABEÇALHO
TABELA DE CABEÇALHO DO PROGRAMA
SEGMENTO 1
SEGMENTO 2
SEGMENTO 3
⋮
TABELA CABEÇALHO DA SEÇÃO (OPCIONAL)

Exercício

- ▶ Realize experimentos para entender melhor a organização dos dados na memória
 - ▶ Analise a representação em memória de estruturas com união (*union*) e campo de bits (*bit fields*)
 - ▶ Entenda os outros segmentos de memória e verifique as diferenças entre o GCC e o LLVM
 - ▶ Pesquise sobre o formato para depuração *Debugging With Attributed Record Formats* (DWARF)